

Introduction to Deodesy

Deodesy is designed to provide .NET developers with a reliable and efficient solution for geographical calculations. Originating as a C#/.NET port of Chris Veness's highly respected JavaScript Geodesy Functions, Deodesy aims to maintain the accuracy and breadth of the original library while integrating seamlessly into the .NET environment.

Why Deodesy?

Geographical calculations are fundamental to many applications, from mapping and navigation to logistics and environmental science. Deodesy addresses the need for a robust, well-tested, and comprehensive geodesy library in C#. By porting a widely-used and validated JavaScript library, Deodesy offers:

- **Proven Accuracy:** Leveraging the algorithms and extensive testing of the original Geodesy Functions.
- **C#/.NET Integration:** Native C# implementation, allowing for easy integration into .NET projects without interoperability overhead.
- **Comprehensive Features:** A rich set of functions covering various aspects of geodesic computations.
- **Maintainability:** Written with modern C# practices, ensuring readability and ease of maintenance.

Core Concepts

Deodesy operates on fundamental geodesic principles. Key concepts you'll encounter include:

- **Coordinates:** Representing points on the Earth's surface using latitude and longitude.
- **Distances:** Calculating the shortest path between two points (great-circle distance).
- **Bearings:** Determining the direction from one point to another.
- **Ellipsoidal vs. Spherical Models:** Understanding the differences between modeling the Earth as a perfect sphere versus a more accurate ellipsoid.

Further Reading

- [Getting Started](#)
- [API Reference](#)
- [Original JavaScript Geodesy Functions by Chris Veness](#) 

Getting Started

Welcome to Deodesy, a robust .NET library for geographical calculations. This guide will help you set up the library and start performing geodesic computations in your applications.

Prerequisites

Before you begin, ensure you have the following installed:

- [.NET SDK](#) (6.0 or later recommended)
- A C# IDE (Visual Studio, JetBrains Rider, or VS Code)

Building from Source

To build the library locally, clone the repository and use the .NET CLI:

```
# Navigate to the library directory
cd src/Deodesy.Library

# Build the project
dotnet build -c Release
```

The compiled binaries will be available in the `src/Deodesy.Library/bin/Release/netstandard2.0` directory.

Target Framework & Compatibility

Deodesy targets *.NET Standard 2.0*, providing maximum reach across the .NET ecosystem.

Platform	Minimum Version
.NET & .NET Core	2.0+
.NET Framework	4.6.1+
Mono	5.4+
Xamarin.iOS / Xamarin.Android	10.14+ / 8.0+

Installation

You can integrate Deodesy into your project by adding a reference to the project file or the compiled DLL.

Via .NET CLI

Run the following command from your project's directory:

```
dotnet add reference path/to/Deodesy.Library.csproj
```

Via Visual Studio

1. Right-click your project in **Solution Explorer**.
2. Select **Add > Project Reference...**
3. Browse and select `Deodesy.Library.csproj`.

Quick Start Example

Here is a simple example demonstrating how to calculate the distance between two geographical points using the spherical model.

```
using Deodesy.Library;

// Define coordinates for London and Washington D.C.
var london = new Coordinate(51.5074, -0.1278);
var washington = new Coordinate(38.8951, -77.0364);

// Initialize the spherical geodesy utility
var geodesy = new LatLonSpherical();

// Calculate distance in kilometers
double distance = geodesy.Distance(london, washington);

Console.WriteLine($"Distance between London and DC: {distance:F2} km");
// Output: Distance between London and DC: 5908.38 km (approx)
```

Next Steps

- Explore the [API Reference](#) for advanced features.